

C++ INTERVIEW NOTES — INDEX

Distilled from *LearnCpp* (foundations, ~C++20) and supplemented with the interview-critical material the book is thin on: the memory model, move-semantics edge cases, template metaprogramming, and STL complexity/perf. Depth level: mechanics + gotchas + code. Each file ends with drill prompts to exercise the topic.

HOW TO READ THESE (LEARNING DEVICES IN EVERY FILE)

Each file is built for *encoding and retrieval*, not just reference. The devices:

- > How to think about this (top of each file) — the mental model to load *before* the facts. Read this first and return to it when a detail confuses you; most details are corollaries of the model.
- > Take-home lesson — the one sentence to remember; the quotable interview answer.
- > War story — a real failure and the principle it teaches (stories stick better than rules).
- Clarification notes — a short boxed note at the trickiest points that states the precise correct model and its concrete consequence (e.g. what `shared_ptr` thread-safety *actually* covers, why `clear()` doesn't deallocate). Read these slowly; they're where the common bugs hide.
- Reason it out — derivations that build a rule from first principles instead of asserting it (e.g. *why* moves must be `noexcept`).
- Stop and think — a mini-problem with a hidden answer (`<details>`). Cover the answer, attempt it out loud, then check. This retrieval step is the single highest-leverage thing in these notes — skipping it turns active learning into passive reading.
- Self-check ladder (end of each file) — four graded levels: Recall → Apply →

Transfer → Teach. You don't know a topic until you can pass Teach.

- Connects to — cross-links to related ideas in other files, so concepts reinforce each other instead of sitting in silos.
- Code blocks use C++17 or newer wherever it's cleaner (CTAD, `if constexpr`, structured bindings, concepts, `jthread`, `expected`, deducing-`this`).

A STUDY PROTOCOL THAT ACTUALLY WORKS (EVIDENCE-BASED)

1. Model first. Read the "How to think about this" box and the section headers only. Predict what the rules will be *before* reading them.
2. Active recall over rereading. At every *Stop and think*, cover the answer and commit to a real answer out loud or on paper. Being wrong then correcting beats reading the right answer cold — that's the *testing effect*.
3. Climb the ladder, don't skim it. Finish a file at Transfer; return a day later and attempt Teach from memory. If you can't teach it, you haven't learned it — reread only the part that failed.
4. Space it. Revisit a file after 1 day, then ~3, then ~7 (spaced repetition). The *Level 1 — Recall* bullets are your quick spaced-review checklist.
5. Interleave. Don't grind one file to perfection before moving on. Rotate files; the *Connects to* links exist so interleaving reinforces rather than confuses. Interleaving feels harder and produces better retention than blocking.
6. Whiteboard the code. For every Apply/Transfer prompt that says "write" or "implement," actually write it by hand before checking. Recognition ≠ recall.

FILES (SUGGESTED FIRST-PASS ORDER)

1. 01 — Memory, Object Lifetime & RAI — storage durations, lifetime rules, RAI, smart pointers, `new` / `delete`, UB catalogue.
2. 02 — Value Categories & Move Semantics — lvalue/xvalue/prvalue, moves, Rule of 0/3/5, copy elision/RVO, perfect forwarding.

3. 03 — Templates & Metaprogramming — instantiation, two-phase lookup, specialization, SFINAE→concepts, `constexpr` / `constexpr`, variadics, CRTP, type erasure.
4. 04 — STL Containers, Algorithms & Perf — complexity table, iterator categories & invalidation, `vector` /hash internals, algorithms, ranges, SSO/`string_view` / `span`, perf mindset.
5. 05 — Concurrency & the Memory Model — threads, mutexes/locks, condition variables, atomics, memory orderings (acquire/release/seq_cst), false sharing, lock-free hazards.
6. 06 — OOP & Polymorphism Internals — vtable/vptr, virtual dtor rule, override/slicing, inheritance flavors & diamond, casts/RTTI, static vs dynamic polymorphism.
7. 07 — Core Language Essentials — const-correctness, references vs pointers, initialization traps, ODR/linkage/ `inline`, exceptions & safety guarantees, `enum class` / `auto`, lambdas.
8. 08 — Modern C++ Map (11→23) — per-standard feature list + the vocabulary types (`optional` / `variant` / `expected` / `span`).

(Order is a starting point, not a track — interleaving beats strict sequence, see protocol above.)

THE FOUR FOCUS AREAS → WHERE THEY LIVE

- Core language + memory/RAII → 01, 07, 06
- Modern C++ (11→23) → 08, 02, 03
- Concurrency & memory model → 05
- Templates/STL internals & perf → 03, 04

HOW TO DRILL (OUR LOOP)

Pick a file → I quiz you from its self-check ladder + follow-ups → you answer / whiteboard code → I probe edge cases and correct. We go until you hit the Teach level. Say e.g. "drill me on file 05" or "quiz: move semantics".

KNOWN COVERAGE GAPS (NOT IN SCOPE OF THESE NOTES, FLAG IF NEEDED)

Coroutines internals, modules/toolchain specifics, allocator design, networking/executors (not yet standard), and deep template-metaprogramming (e.g. full `tuple`/`variant` implementations) — we can add a file if a target role emphasizes any of these.